

**CURSO DE ENGENHARIA DE SOFTWARE**

**Fundamentos de Banco de Dados**

**Sistema de Fórum de discussão Edu Help**

**Aluno:**

Francisco Josias Da Silva Batista - 542167

Rafael Sousa Cabral - 542172

**Professora:**

Livia Almada Cruz

**Quixadá - CE**

**Dezembro, 2023**

## **1. Sistema**

### **1.1 Título do sistema:**

EduHelp

### **1.2 Descrição do sistema:**

A aplicação “EduHelp” tem como objetivo criar um ambiente online onde estudantes possam interagir, fazer perguntas, compartilhar conhecimentos, e discutir tópicos sobre as disciplinas e dúvidas. Esta plataforma fornece uma maneira conveniente e eficaz para os estudantes aprenderem, colaborarem e se atualizarem em um ambiente amigável e construtivo.

### **1.3 Entidades Envolvidas:**

#### **1.3.1 Usuário:**

Atributos: id do usuário(chave primária), nome, sobrenome, telefone, senha, email, data de registro.

Relacionamentos: Os usuários podem criar tópicos, postar mensagens e comentários nos tópicos. Um usuário pode ter várias postagens e comentários.

#### **1.3.2 Tópicos:**

Atributos: id do tópico(chave primária), título, descrição, autor(id de usuário).

Relacionamentos: Os tópicos de discussão são criados por usuários. Cada tópico de discussão pode ter várias mensagens relacionadas. Agora o atributo de data de criação está no relacionamento de criar tópicos.

#### **1.3.3 Mensagens:**

Atributos: id da mensagem(chave primária), conteúdo da mensagem, data de postagem, autor(id do usuário), tópico de discussão relacionado(id do tópico).

Relacionamentos: As mensagens são postadas por usuários em tópicos de discussão específicos. Cada tópico de discussão pode ter várias mensagens.

#### **1.3.4 Comentários:**

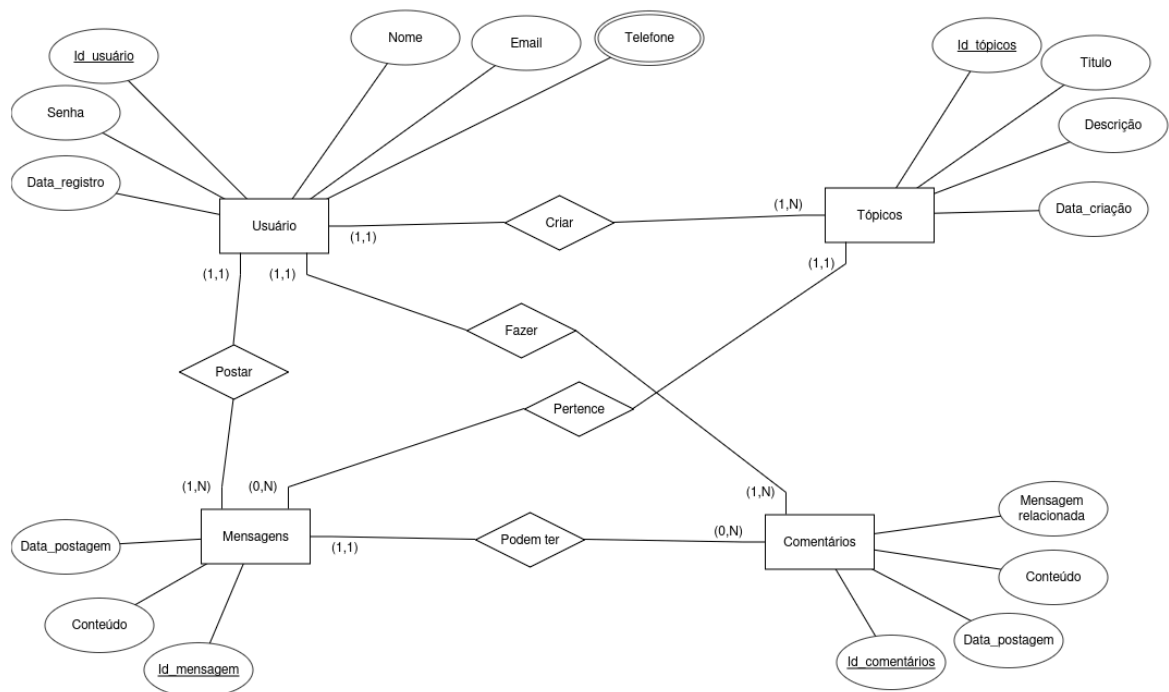
Atributos: id do comentário(chave primária), conteúdo do comentário, data de postagem, autor(id de usuário), mensagem relacionada, id da mensagem.

Relacionamentos: Os comentários podem ser adicionados às mensagens existentes. Cada mensagem pode ter vários comentários (ou nenhum).

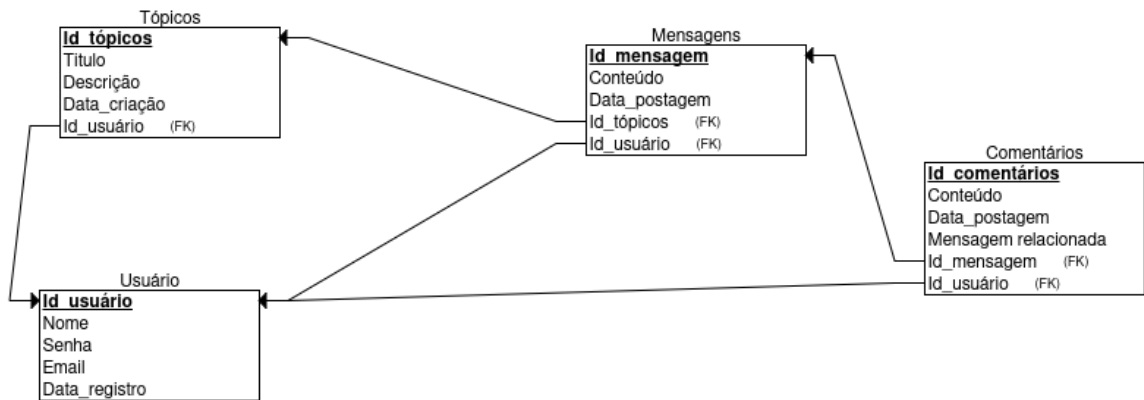
#### 1.4 Restrições de cardinalidade:

- Um usuário pode criar vários tópicos de discussão, postar várias mensagens e fazer vários comentários.
- Cada tópico de discussão é criado por um único usuário.
- Cada mensagem é postada por um único usuário e está associada a um único tópico de discussão.
- Os comentários podem ser feitos por um único usuário e estão relacionados a uma única mensagem.

#### 1.5 Modelo de entidade relacionamento(ER/EER):



## 1.6 Modelo Relacional:



## 1.7 Implementação do Banco de dados:

### 1.7.1 Esquema do Banco de Dados:

A) criação de tabelas código abaixo:

**CREATE TABLE usuario**

```

(
  id_usuario SERIAL,
  nome VARCHAR(100) NOT NULL,
  sobrenome VARCHAR(100) NOT NULL,
  email VARCHAR(255) UNIQUE,
  senha VARCHAR(100) NOT NULL,
  data_registro DATE,
  PRIMARY KEY (id_usuario)
);
  
```

**CREATE TABLE topicos**

```

(
  
```

```
id_topicos SERIAL,  
id_usuario INT NOT NULL,  
titulo VARCHAR(100) NOT NULL,  
descricao TEXT NOT NULL,  
ultima_modificacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
contador_mensagens INT DEFAULT 0  
PRIMARY KEY (id_topicos),  
FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)  
);
```

**CREATE TABLE mensagens**

```
(  
    id_mensagem SERIAL,  
    id_topicos INT NOT NULL,  
    id_usuario INT NOT NULL,  
    conteudo TEXT NOT NULL,  
    data_postagem DATE,  
    PRIMARY KEY (id_mensagem),  
    FOREIGN KEY (id_topicos) REFERENCES topicos(id_topicos),  
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)  
);
```

**CREATE TABLE comentarios**

```
(  
    id_comentario SERIAL,  
    id_mensagem INT NOT NULL,  
    id_usuario INT NOT NULL,  
    conteudo TEXT NOT NULL,  
    data_postagem DATE,
```

```

mensagem_relacionada INT NOT NULL,
PRIMARY KEY (id_comentario),
FOREIGN KEY (id_mensagem) REFERENCES mensagens(id_mensagem),
FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
);

```

**B) Povoar o Banco com dados (pelo menos 10 tuplas em cada relação ):**

apagar todos os dados.

```

TRUNCATE TABLE usuario RESTART IDENTITY CASCADE;
TRUNCATE TABLE topicos RESTART IDENTITY CASCADE;
TRUNCATE TABLE mensagens RESTART IDENTITY CASCADE;
TRUNCATE TABLE comentarios RESTART IDENTITY CASCADE;

```

**INSERT INTO usuario (nome, sobrenome, email, senha, data\_registro)**

**VALUES**

```

('Estudante1', 'Sobrenome1', 'estudante1@example.com', 'senha123', CURRENT_DATE),
('Estudante2', 'Sobrenome2', 'estudante2@example.com', 'senha456', CURRENT_DATE),
('Estudante3', 'Sobrenome3', 'estudante3@example.com', 'senha789', CURRENT_DATE),
('Estudante4', 'Sobrenome4', 'estudante4@example.com', 'senhaabc', CURRENT_DATE),
('Estudante5', 'Sobrenome5', 'estudante5@example.com', 'senhade', CURRENT_DATE),
('Estudante6', 'Sobrenome6', 'estudante6@example.com', 'senhaghi', CURRENT_DATE),
('Estudante7', 'Sobrenome7', 'estudante7@example.com', 'senhajkl', CURRENT_DATE),
('Estudante8', 'Sobrenome8', 'estudante8@example.com', 'senhamno', CURRENT_DATE),
('Estudante9', 'Sobrenome9', 'estudante9@example.com', 'senhapqr', CURRENT_DATE),
('Estudante10', 'Sobrenome10', 'estudante10@example.com', 'senhastu',
CURRENT_DATE)
RETURNING id_usuario;

```

**INSERT INTO topicos (id\_usuario, titulo, descricao)**

**VALUES**

(1, 'Introdução aos Bancos de Dados', 'Discussão sobre conceitos básicos de bancos de dados.'),

(2, 'Modelagem de Dados', 'Tópico sobre a criação de modelos de dados.'),

(3, 'SQL e Consultas', 'Discussão sobre a linguagem SQL e consultas em bancos de dados.'),

(4, 'Normalização de Dados', 'Tópico sobre o processo de normalização de dados em bancos de dados.'),

(5, 'Bancos de Dados Relacionais', 'Explorando bancos de dados relacionais.'),

(6, 'Bancos de Dados NoSQL', 'Discussão sobre bancos de dados NoSQL.'),

(7, 'Administração de Bancos de Dados', 'Tópico sobre a administração de sistemas de bancos de dados.'),

(8, 'Recuperação de Dados', 'Como recuperar dados de um banco de dados.'),

(9, 'Backup e Restauração', 'Estratégias de backup e restauração em bancos de dados.'),

(10, 'Segurança de Dados', 'Proteção de dados em sistemas de banco de dados.');

**INSERT INTO mensagens (id\_topicos, id\_usuario, conteudo, data\_postagem)**

**VALUES**

(1, 1, 'Vamos estudar os conceitos básicos de bancos de dados.', '2023-10-02'),

(2, 2, 'Modelagem de dados é crucial para o design de bancos de dados eficazes.', '2023-10-03'),

(3, 3, 'Qual é a sua consulta SQL favorita?', '2023-10-04'),

(4, 4, 'Normalização é importante para evitar redundância de dados.', '2023-10-05'),

(5, 1, 'Vamos discutir bancos de dados relacionais.', '2023-10-06'),

```
(6, 2, 'Você já trabalhou com bancos de dados NoSQL?', '2023-10-07'),
(7, 3, 'Dicas para administrar bancos de dados de grande escala.', '2023-10-08'),
(8, 4, 'Como recuperar dados perdidos em um banco de dados.', '2023-10-09'),
(9, 5, 'Estratégias de backup são essenciais para a recuperação de dados.', '2023-10-10'),
(10, 6, 'Segurança de dados é uma prioridade em bancos de dados.', '2023-10-11');
```

```
INSERT INTO comentarios (id_mensagem, id_usuario, conteudo, data_postagem,
mensagem_relacionada)
```

```
VALUES
```

```
(1, 5, 'Eu concordo, vamos aprender sobre bancos de dados relacionais.', '2023-10-03',
21),
(2, 6, 'Modelagem de dados é a base para um banco de dados eficiente.', '2023-10-04',
22),
(3, 7, 'Eu gosto de consultas SQL que envolvem junções.', '2023-10-05', 23),
(4, 8, 'Normalização é fundamental para manter a integridade dos dados.', '2023-10-06',
24),
(5, 9, 'Bancos de dados relacionais são amplamente utilizados na indústria.', '2023-10-07',
25),
(6, 10, 'NoSQL é interessante para aplicações escaláveis.', '2023-10-08', 26),
(7, 1, 'Administrar bancos de dados grandes pode ser desafiador.', '2023-10-09', 27),
(8, 2, 'Recuperação de dados é uma habilidade importante para administradores de
bancos de dados.', '2023-10-10', 28),
(9, 3, 'Backup regular é crucial para evitar perda de dados.', '2023-10-11', 29),
(10, 4, 'A segurança de dados deve ser prioridade em qualquer aplicação.', '2023-10-12',
30);
```

## **1.8 10 Perguntas sobre o Banco de Dados:**



**1.8.1 Quantos comentários foram postados por cada usuário?**

```
SELECT usuario.nome, COUNT(c.id_comentario) AS total_comentarios
FROM usuario
LEFT JOIN comentarios c ON usuario.id_usuario = c.id_usuario
GROUP BY usuario.nome;
```

**1.8.2 Qual é o tópico mais popular com base no número de mensagens associadas?**

```
SELECT topicos.titulo, COUNT(mensagens.id_mensagem) AS total_mensagens
FROM topicos
LEFT JOIN mensagens ON topicos.id_topicos = mensagens.id_topicos
GROUP BY topicos.titulo
ORDER BY total_mensagens DESC
LIMIT 1;
```

**1.8.3 Qual é a média de comentários por mensagem?**

```
SELECT          AVG(comentarios_por_mensagem)          AS
media_comentarios_por_mensagem

FROM          (SELECT          COUNT(comentarios.id_comentario)          AS
comentarios_por_mensagem
FROM mensagens
LEFT JOIN comentarios ON mensagens.id_mensagem =
comentarios.id_mensagem
GROUP BY mensagens.id_mensagem) AS subquery;
```

**1.8.4 Quais são os usuários que mais postaram mensagens no tópico segurança de dados?**

```
SELECT          usuario.nome,          COUNT(mensagens.id_mensagem)          AS
total_mensagens
FROM usuario
INNER JOIN mensagens ON usuario.id_usuario = mensagens.id_usuario
WHERE mensagens.id_topicos = 20
GROUP BY usuario.nome
ORDER BY total_mensagens DESC;
```

**1.8.5 Qual é a média de caracteres por mensagem para cada usuário?**

```
SELECT usuario.nome, AVG(LENGTH(mensagens.conteudo)) AS
media_caracteres_por_mensagem
FROM usuario
INNER JOIN mensagens ON usuario.id_usuario = mensagens.id_usuario
GROUP BY usuario.nome;
```

**1.8.6 Quais são os tópicos sem nenhum comentário associado?**

```
SELECT topicos.titulo
FROM topicos
LEFT JOIN mensagens ON topicos.id_topicos = mensagens.id_topicos
LEFT JOIN comentarios ON mensagens.id_mensagem =
comentarios.id_mensagem
WHERE comentarios.id_comentario IS NULL;
```

**1.8.7 Qual é o usuário que postou o comentário mais longo (em termos de caracteres)?**

```
SELECT usuario.nome, MAX(LENGTH(comentarios.conteudo)) AS
comprimento_maximo
FROM usuario
INNER JOIN comentarios ON usuario.id_usuario = comentarios.id_usuario
GROUP BY usuario.nome
ORDER BY comprimento_maximo DESC
LIMIT 1;
```

**1.8.8 Qual é a mensagem com o maior número de comentários?**

```
SELECT mensagens.conteudo, COUNT(comentarios.id_comentario) AS
total_comentarios
FROM mensagens
LEFT JOIN comentarios ON mensagens.id_mensagem =
comentarios.id_mensagem
GROUP BY mensagens.conteudo
ORDER BY total_comentarios DESC
```

LIMIT 1;

#### 1.8.9 Qual é a média de comentários por tópico?

```
SELECT AVG(total_comentarios) AS media_comentarios_por_topico
FROM (SELECT topicos.id_topicos, COUNT(comentarios.id_comentario) AS
      total_comentarios
FROM topicos
LEFT JOIN mensagens ON topicos.id_topicos = mensagens.id_topicos
LEFT JOIN comentarios ON mensagens.id_mensagem =
      comentarios.id_mensagem
GROUP BY topicos.id_topicos) AS subquery;
```

#### 1.8.10 Qual é o tópico mais popular em termos de atividade recente (mensagens ou comentários postados nas últimas 24 horas)?

```
SELECT topicos.titulo, COUNT(DISTINCT mensagens.id_mensagem) +
COUNT(DISTINCT comentarios.id_comentario) AS atividade_recente
FROM topicos
LEFT JOIN mensagens ON topicos.id_topicos = mensagens.id_topicos AND
      mensagens.data_postagem >= NOW() - INTERVAL '1 day'
LEFT JOIN comentarios ON mensagens.id_mensagem =
      comentarios.id_mensagem AND comentarios.data_postagem >= NOW() -
INTERVAL '1 day'
GROUP BY topicos.titulo
ORDER BY atividade_recente DESC
LIMIT 1;
```

#### VIEWS SQL:

```
CREATE VIEW resumo_topico AS
```

```
SELECT
```

```
  t.id_topicos AS topico_id,
```

```
  t.id_usuario AS autor_id,
```

```
  t.titulo AS topico_titulo,
```

```
  t.descricao AS topico_descricao,
```

```

m.id_mensagem AS mensagem_id,
m.id_usuario AS mensagem_autor_id,
m.conteudo AS mensagem_conteudo,
m.data_postagem AS mensagem_data_postagem,
c.id_comentario AS comentario_id,
c.id_usuario AS comentario_autor_id,
c.conteudo AS comentario_conteudo,
c.data_postagem AS comentario_data_postagem,
c.mensagem_relacionada AS comentario_mensagem_relacionada
FROM
  topicos t
  LEFT JOIN mensagens m ON t.id_topicos = m.id_topicos
  LEFT JOIN comentarios c ON m.id_mensagem = c.id_mensagem;

```

### Trigger:

#### **Contador de mensagens nos tópicos:**

```
-- Incrementa o contador de mensagens ao adicionar uma nova mensagem
```

```
ALTER TABLE topicos
```

```
ADD COLUMN IF NOT EXISTS contador_mensagens INT DEFAULT 0;
```

```
CREATE OR REPLACE FUNCTION atualizar_contador_mensagens()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
-- Incrementa o contador de mensagens ao adicionar uma nova mensagem
```

```
UPDATE topicos
```

```
SET contador_mensagens = contador_mensagens + 1
```

```
WHERE id_topicos = NEW.id_topicos;
```

```
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trigger_atualizar_contador  
AFTER INSERT  
ON mensagens  
FOR EACH ROW  
EXECUTE FUNCTION atualizar_contador_mensagens();
```

### **Salvar a ultima modificação dos tópicos:**

```
ALTER TABLE topicos  
ADD COLUMN ultima_modificacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
```

```
CREATE OR REPLACE FUNCTION atualizar_ultima_modificacao()  
RETURNS TRIGGER AS $$  
BEGIN  
    NEW.ultima_modificacao = CURRENT_TIMESTAMP;  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trigger_atualizar_ultima_modificacao  
BEFORE UPDATE ON topicos  
FOR EACH ROW  
EXECUTE FUNCTION atualizar_ultima_modificacao();
```

